

Threads API

CS232 Operating Systems, Fall 2019

Taj Muhammad Khan

Thread creation

```
#include <pthread.h>
int
pthread_create(pthread_t      *thread,
               const pthread_attr_t *attr,
               void            * (*start_routine) (void*),
               void            *arg);
```

Thread completion

```
int pthread_join(pthread_t thread, void **value_ptr);
```

Threads – passing arguments

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  typedef struct {
5      int a;
6      int b;
7  } myarg_t;
8
9  void *mythread(void *arg) {
10     myarg_t *args = (myarg_t *) arg;
11     printf("%d %d\n", args->a, args->b);
12     return NULL;
13 }
14
15 int main(int argc, char *argv[]) {
16     pthread_t p;
17     myarg_t args = { 10, 20 };
18
19     int rc = pthread_create(&p, NULL, mythread, &args);
20     ...
21 }
```

Threads – returning values

```
1  typedef struct { int a; int b; } myarg_t;
2  typedef struct { int x; int y; } myret_t;
3
4  void *mythread(void *arg) {
5      myret_t *rvals = Malloc(sizeof(myret_t));
6      rvals->x = 1;
7      rvals->y = 2;
8      return (void *) rvals;
9  }
10
11 int main(int argc, char *argv[]) {
12     pthread_t p;
13     myret_t *rvals;
14     myarg_t args = { 10, 20 };
15     Pthread_create(&p, NULL, mythread, &args);
16     Pthread_join(p, (void **) &rvals);
17     printf("returned %d %d\n", rvals->x, rvals->y);
18     free(rvals);
19     return 0;
20 }
```

Threads – returning values

```
1 void *mythread(void *arg) {  
2     myarg_t *args = (myarg_t *) arg;  
3     printf("%d %d\n", args->a, args->b);  
4     myret_t oops; // ALLOCATED ON STACK: BAD!  
5     oops.x = 1;  
6     oops.y = 2;  
7     return (void *) &oops;  
8 }
```

Locks